

OpenCode / Cheat Sheet

Agente de IA **open source** para programar desde la terminal, el escritorio o tu IDE.
Instalación, comandos de CLI, atajos del TUI, configuración y flujo de trabajo en una sola referencia.

- Docs · [opencode.ai/docs](#)
- GitHub · [anomalyco/opencode](#)
- Modelos · [models.dev](#)

● **INSTALACIÓN**

SETUP

SCRIPT UNIVERSAL · RECOMENDADO

```
curl -fsSL https://opencode.ai/install | bash
```

NODE.JS

```
npm install -g opencode-ai
```

MACOS / LINUX · HOMEBREW

```
brew install anomalyco/tap/opencode
```

ARCH LINUX

```
sudo pacman -S opencode          # Stable
paru -S opencode-bin             # AUR (latest)
```

WINDOWS

```
choco install opencode           # Chocolatey
scoop install opencode           # Scoop
npm install -g opencode-ai       # NPM
mise use -g github:anomalyco/opencode
docker run -it --rm \
  ghcr.io/anomalyco/opencode
```

! En **Windows** se recomienda usar **WSL** para mejor compatibilidad.

● **ACTUALIZAR / DESINSTALAR**

START

opencode upgrade # Última versión
opencode upgrade v0.1.48 # Versión
específica
opencode uninstall # Desinstalar
opencode uninstall --keep-config # Conservar config

● **PRIMEROS PASOS**

```
cd /ruta/a/mi/proyecto
opencode          # Inicia el TUI
/connect          # Conectar proveedor de IA
/init             # Analiza el proyecto → AGENTS.md
```

git

Commitea **AGENTS.md** a Git para que el agente entienda tu proyecto en cada sesión.

● **ARRANCA CON BUEN PIE**

TIPS

▶ **Plan primero:** pulsa `Tab` para diseñar el enfoque y revísalo antes de pasar a **Build**.

▶ **Retoma rápido:** `opencode -c` continúa tu última sesión.

▶ **Sin miedo:** `/undo` revierte el mensaje y los cambios de archivos.

▶ **Contexto persistente:** ejecuta `/init` y commitea `AGENTS.md`.

▶ **Ahorra tokens:** define un `small_model` (p. ej. Haiku) para tareas ligeras.

▶ **Multimodelo:** cambia de modelo en marcha con `/models` o `F2`.

opencode.ai

Hoja de atajos · 01 / 08

cursos.devtales.com

● TUI · ARRANQUE

CLI

```
opencode                # TUI en dir actual
opencode /ruta/proyecto  # TUI en dir específico
opencode -c              # Continuar última sesión
opencode -s <sessionID>  # Continuar sesión por ID
opencode -m anthropic/claude-sonnet-4-5
```

● MODO NO-INTERACTIVO

```
opencode run "Explica los closures en JS"
opencode run -m openai/gpt-4o "Refactoriza"
opencode run --share "Mensaje" # Auto-compartir
opencode run -f archivo.ts "Revisa esto"
```

● SERVIDOR HEADLESS

```
opencode serve # API sin TUI (puerto random)
opencode web   # Servidor + UI web en browser
opencode attach http://10.0.0.1:4096
```

● AUTH / PROVEEDORES

```
opencode auth login # Iniciar sesión
opencode auth login -p anthropic # Proveedor
opencode auth list  # Listar
opencode auth logout # Cerrar sesión
```

● MODELOS

```
opencode models # Listar todos
opencode models anthropic # Filtrar por proveedor
opencode models --refresh # Refrescar caché
opencode models --verbose # Metadata (costos...)
```

● SESIONES

```
opencode session list # Listar
opencode session list -n 10 # Últimas 10
opencode session delete <id>
opencode export <sessionID> # → JSON
opencode import session.json
opencode import https://opncd.ai/s/abc123
```

● MCP · AGENTES · PLUGINS

```
opencode mcp add / list # Servidores MCP
opencode mcp auth <nombre> # OAuth servidor
opencode mcp logout <nombre>
opencode agent create / list # Agentes
opencode plugin <modulo> # Plugin (-g global)
```

● OTROS

```
opencode stats # Tokens y costos
opencode stats --days 7 # Últimos 7 días
opencode pr <número> # Checkout PR + opencode
opencode db path # Ruta de la base de datos
```

● FLAGS GLOBALES

FLAGS

FLAG	ALIAS	DESCRIPCIÓN
<code>--help</code>	<code>-h</code>	Mostrar ayuda
<code>--version</code>	<code>-v</code>	Versión instalada
<code>--log-level</code>	—	DEBUG / INFO / WARN / ERROR
<code>--print-logs</code>	—	Imprimir logs a stderr
<code>--pure</code>	—	Ejecutar sin plugins externos

● COMANDOS DE BARRA /

TUI

COMANDO	ACCIÓN
<code>/init</code>	Crear/actualizar AGENTS.md
<code>/connect</code>	Agregar proveedor de IA
<code>/new</code> <code>/clear</code>	Nueva sesión
<code>/sessions</code>	Listar / cambiar sesión
<code>/models</code>	Ver modelos disponibles
<code>/undo</code>	Deshacer mensaje + cambios
<code>/redo</code>	Rehacer mensaje deshecho
<code>/compact</code>	Compactar sesión actual
<code>/share</code>	Compartir (copia link)
<code>/unshare</code>	Dejar de compartir
<code>/export</code>	Exportar a Markdown
<code>/editor</code>	Editor externo para prompt
<code>/themes</code>	Ver temas disponibles
<code>/thinking</code>	Toggle razonamiento
<code>/details</code>	Toggle detalles de tools
<code>/help</code>	Mostrar ayuda
<code>/exit</code> <code>/q</code>	Salir

● REFERENCIAS EN EL PROMPT

```
@nombre-archivo # Fuzzy de archivos del proyecto
!ls -la          # Shell (output → contexto)
```

+ Arrastra y suelta **imágenes** al terminal para adjuntarlas.

● ATAJOS · NAVEGACIÓN

KEYS

<code>Ctrl</code> + <code>P</code>	Paleta de comandos
<code>Tab</code>	Modo Build ↔ Plan
<code>Shift</code> + <code>Tab</code>	Ciclar modo en reversa
<code>Ctrl</code> + <code>T</code>	Variante del modelo
<code>Esc</code>	Interrumpir generación
<code>Enter</code>	Enviar mensaje
<code>Shift</code> + <code>Enter</code>	Nueva línea
<code>PgUp</code> / <code>PgDn</code>	Scroll por mensajes
<code>Home</code> / <code>End</code>	Inicio / final
<code>F2</code>	Ciclar modelo reciente

● LEADER KEY · `CTRL+X`

KEYS

☞ Pulsa la **leader** `Ctrl` + `X` y luego la tecla de acción.

<code>⌘X</code> → <code>N</code>	Nueva sesión
<code>⌘X</code> → <code>C</code>	Compactar sesión
<code>⌘X</code> → <code>U</code>	Undo
<code>⌘X</code> → <code>R</code>	Redo
<code>⌘X</code> → <code>L</code>	Listar sesiones
<code>⌘X</code> → <code>M</code>	Listar modelos
<code>⌘X</code> → <code>T</code>	Listar temas
<code>⌘X</code> → <code>E</code>	Editor externo
<code>⌘X</code> → <code>X</code>	Exportar conversación
<code>⌘X</code> → <code>Y</code>	Copiar mensajes
<code>⌘X</code> → <code>B</code>	Mostrar / ocultar barra lateral
<code>⌘X</code> → <code>Q</code>	Salir

● ATAJOS · EDICIÓN DE INPUT

<code>Ctrl</code> + <code>A</code>	Inicio de línea
<code>Ctrl</code> + <code>E</code>	Final de línea
<code>Ctrl</code> + <code>K</code>	Borrar hasta fin de línea
<code>Ctrl</code> + <code>U</code>	Borrar hasta inicio
<code>Ctrl</code> + <code>W</code>	Borrar palabra anterior

● TIPS DEL TUI

TIPS

- ▶ `@archivo` hace fuzzy-search; `!comando` inyecta la salida del shell al contexto.
- ▶ Arrastra **imágenes** al terminal para adjuntarlas a tu prompt.
- ▶ `Ctrl+P` abre la paleta: ejecuta cualquier acción sin memorizar atajos.
- ▶ Usa `/compact` en sesiones largas para resumir y ahorrar tokens.

● ARCHIVOS DE CONFIGURACIÓN

CONFIG

ARCHIVO	UBICACIÓN	PROPÓSITO
opencode.json	~/.config/opencode/	Config global (servidor/runtime)
opencode.json	Raíz del proyecto	Config por proyecto
tui.json	~/.config/opencode/	Config visual del TUI
AGENTS.md	Raíz del proyecto	Instrucciones del agente
auth.json	~/.local/share/opencode/	Credenciales de proveedores

 Las configs se **fusionan**, no se reemplazan. El proyecto tiene prioridad sobre la global.

● OPENCODE.JSON

```
{
  "$schema": "https://opencode.ai/config.json",
  "model": "anthropic/claude-sonnet-4-5",
  "small_model": "anthropic/claude-haiku-4-5",
  "autoupdate": true,
  "share": "manual",
  "default_agent": "build",
  "permission": { "edit": "ask", "bash": "ask" },
  "compaction": { "auto": true, "prune": false },
  "formatter": true,
  "lsp": true,
  "snapshot": true
}
```

● TUI.JSON

```
{
  "$schema": "https://opencode.ai/tui.json",
  "theme": "opencode",
  "keybinds": {
    "leader": "ctrl+x",
    "command_list": "ctrl+p"
  },
  "scroll_speed": 3,
  "diff_style": "auto",
  "mouse": true,
  "attention": {
    "enabled": true, "sound": true, "volume": 0.4
  }
}
```

● VARIABLES DE ENTORNO

OPENCODE_CONFIG	Ruta a config personalizada
OPENCODE_CONFIG_DIR	Directorio de config
OPENCODE_CONFIG_CONTENT	Config JSON inline
OPENCODE_AUTO_SHARE	Compartir sesiones auto
OPENCODE_SERVER_PASSWORD	Auth básica del servidor
OPENCODE_DISABLE_AUTOUPDATE	Sin actualizaciones auto
OPENCODE_DISABLE_AUTOCOMPACT	Sin compactación auto
OPENCODE_PERMISSION	Permisos JSON inline
EDITOR	Editor externo (vim, code--wait)

● SUSTITUCIÓN DE VARIABLES

```
{
  "model": "{env:OPENCODE_MODEL}",
  "provider": {
    "anthropic": {
      "options": {
        "apiKey": "{env:ANTHROPIC_API_KEY}"
      }
    },
    "openai": {
      "options": {
        "apiKey": "{file:~/.secrets/openai-key}"
      }
    }
  }
}
```

 Usa **{env:VAR}** para variables de entorno y **{file:ruta}** para leer secretos desde un archivo.

● MODOS INTEGRADOS

AGENTS

MODO	DESCRIPCIÓN
Build	Default. Lee y modifica archivos
Plan	Solo sugiere, no modifica

➡ Cambia entre modos con `Tab` dentro del TUI.

● AGENTE POR CLI

```
opencode agent create \  
  --description "Revisor de PRs" \  
  --mode primary \  
  --permissions "read,grep,glob" \  
  --model anthropic/claude-sonnet-4-5
```

● AGENTE EN MARKDOWN · `.OPENCODE/AGENTS/MI-AGENTE.MD`

```
---  
description: Revisor de código enfocado en seguridad  
mode: subagent  
model: anthropic/claude-sonnet-4-5  
permission:  
  edit: deny  
  bash: deny  
---
```

Eres un revisor de código. Enfócate en seguridad, rendimiento y mantenibilidad.

! El campo `tools` (`true` / `false`) está **deprecado**; usa `permission` con `allow` · `ask` · `deny` .

● TIPS & BUENAS PRÁCTICAS

TIPS

- ▶ **Permisos:** `"edit":"ask"` y `"bash":"ask"` te dejan aprobar cada acción.
- ▶ **Plan mode** brilla en refactors grandes: acuerda el plan y ejecútalo en Build.
- ▶ **Comparte:** `/share` genera un link para revisar la sesión en equipo.
- ▶ **Sin perder hilo:** activa `compaction.auto` en sesiones largas.
- ▶ **Agentes a medida:** restringe con `permission` (`edit: deny`) para roles de solo lectura.
- ▶ **Agente por defecto:** fíjalo con `default_agent` en `opencode.json` .
- ▶ **Subagentes:** la tool `task` delega trabajo en paralelo.
- ▶ **Documenta:** `/export` guarda la conversación en Markdown.

● PROMPTS DE EJEMPLO

PROMPTS

- › `@src/auth.ts` explica el flujo de login y posibles vulnerabilidades
- › Refactoriza este módulo a `async/await` y añade pruebas
- › `!git diff` resume los cambios y redacta el commit
- › Genera tests unitarios con casos borde para esta función
- › Crea un endpoint REST `/users` con validación y errores
- › Documenta este archivo con comentarios JSDoc claros

HERRAMIENTAS (TOOLS)

TOOLS

bash

Ejecutar comandos shell

read

edit

Leer / modificar · reemplazo exacto

write

apply_patch

Crear / sobrescribir · parchear

glob

grep

Buscar archivos / contenido

webfetch

websearch

Obtener URLs / buscar web

todowrite

skill

Lista de tareas / ejecutar skills

question

Preguntar al usuario en la tarea

task

Delegar a subagentes

lsp

Diagnósticos de lenguaje · experimental

```
# Control vía permisos · "tools" quedó deprecado
# "edit" cubre write · edit · apply_patch
{ "permission": {
  "edit": "ask", "bash": "ask", "webfetch":
  "allow"
} }
```

MCP SERVERS

MCP

```
{
  "mcp": {
    "mi-servidor": {
      "type": "local",
      "command": ["npx", "-y",
        "@modelcontextprotocol/server-filesystem"],
      "args": ["/ruta/al/proyecto"]
    },
    "servidor-remoto": {
      "type": "remote",
      "url": "https://mcp.ejemplo.com",
      "enabled": true
    }
  }
}
```

FLUJO RECOMENDADO

WORKFLOW

```
1 cd mi-proyecto
2 opencode
3 /connect      → autenticar proveedor
4 /init         → analizar → AGENTS.md
5 Tab          → cambiar a Plan
6 "Describe la feature que quieres"
7 Tab          → volver a Build
8 "Implementa el plan"
9 /undo         → si algo salió mal
10 /share       → compartir con el equipo
```

TIPS DE AUTOMATIZACIÓN

TIPS

- ▶ **Modo headless:** `opencode run "..."` en scripts, hooks de Git y CI.
- ▶ **Seguridad:** usa `"bash": "ask"` o `"deny"` en entornos sensibles.
- ▶ **Extiende con MCP:** conecta servidores para dar acceso a tus datos y herramientas.
- ▶ **Vigila costos:** `opencode stats --days 7` muestra tokens y gasto.
- ▶ **UI web:** `opencode web` abre una interfaz sin salir del flujo.
- ▶ **Remoto:** `opencode serve` + `attach` contra un servidor.

LINKS ÚTILES

LINKS

Documentación · `opencode.ai/docs`

Modelos · `models.dev`

Discord · `opencode.ai/discord`

Changelog · `opencode.ai/changelog`

Proveedores · `/docs/providers`

Zen · `opencode.ai/zen`

GitHub · `anomalyco/opencode`

Cursos · `cursos.devttalles.com`

● REFERENCIAS · DIRECTORIOS EXTERNOS

REFERENCES

¿QUÉ SON?

Dan a los agentes y a la TUI acceso a **directorios y repos Git fuera del proyecto** (docs, librerías, ejemplos). Se definen por **alias** en `opencode.json` y se invocan con `@alias`.

EJEMPLO BÁSICO

```
{ "references": {
  "docs": { "path": "../product-docs" },
  "sdk": {
    "repository": "anomalyco/opencode-sdk-js",
    "branch": "main"
  }
}
```

● DIRECTORIOS LOCALES

PATH

● REPOSITARIOS GIT

REPOSITORY

```
{ "references": {
  "docs": { "path": "../docs" }
} }
```

Forma corta (solo path)

```
{ "references": { "docs": "../docs" } }
```

<code>../docs</code>	Relativo al archivo de config
<code>/home/user/docs</code>	Ruta absoluta
<code>~/docs</code>	Relativo al home del usuario

```
{ "references": {
  "effect": {
    "repository": "Effect-TS/effect",
    "branch": "main"
  }
} }
```

Forma corta (sin branch)

```
{ "references": { "effect": "Effect-TS/effect" } }
```

i

`repository` acepta URL Git, `host/path` o `owner/repo`. `branch` es opcional. Se clona/refresca de forma **asíncrona**.

● DESCRIPTION & HIDDEN

FLAGS

```
{ "design-system": {
  "path": "../design-system",
  "description": "Componentes UI y tokens"
} }
```

+

Con `description` la referencia se **incluye en el contexto**; sin ella sigue por autocomplete y uso directo, pero no se anuncia.

```
{ "internal": {
  "path": "../internal",
  "description": "Detalles internos",
  "hidden": true
} }
```

!

`hidden` solo oculta del autocomplete `@`. Con `description`, **sigue en el contexto**.

● USO EN LA TUI · CAMPOS & ALIAS

USAGE

<code>@alias</code>	Adjunta la raíz de la referencia
<code>@alias/</code>	Busca archivos dentro de ella

Compara esto con `@sdk/src/client.ts`

►

Los agentes con `description` reciben las rutas sin adjuntarlas a mano (external-directory permission boundary).

►

Los **permisos de tools** siguen aplicando: la referencia no concede edición.

CAMPO	ÁMBITO	DESCRIPCIÓN
<code>path</code>	Local	Directorio local
<code>repository</code>	Git	<code>owner/repo</code> , URL o host/path
<code>branch</code>	Git	Rama o ref opcional
<code>description</code>	Ambos	Cuándo usar la referencia
<code>hidden</code>	Ambos	Oculta del autocomplete <code>@</code>

!

Los **alias** no pueden estar vacíos ni contener `/`, espacios, backticks ni comas.

opencode.ai/docs

Referencias · 07 / 08

cursos.devtales.com

● CUSTOM TOOLS · FUNCIONES QUE EL LLM INVOKA

TOOLS

Funciones propias que el modelo puede llamar en una conversación, junto a las built-in (`read`, `write`, `bash`). Se **definen** en TS/JS, pero su lógica puede correr en **cualquier lenguaje**.

UBICACIÓN

<code>.opencode/tools/</code>	Por proyecto
<code>~/.config/opencode/tools/</code>	Global

i El **nombre del archivo** es el nombre del tool: `database.ts` → tool `database`.

ESTRUCTURA BÁSICA · DATABASE.TS

```
import { tool } from "@opencode-ai/plugin"
export default tool({
  description: "Query the project database",
  args: { query: tool.schema.string().describe("SQL") },
  async execute(args) {
    return `Ran: ${args.query}`
  },
})
```

● VARIAS TOOLS POR ARCHIVO

```
// math.ts
export const add = tool({
  description: "Add two numbers",
  args: { a: tool.schema.number(), b:
tool.schema.number() },
  async execute(a) { return (a.a + a.b).toString() },
})
export const multiply = tool({ /* ... */ })
```

+ Cada export genera `archivo_export`: aquí `math_add` y `math_multiply`.

● COLISIÓN CON BUILT-IN

```
// bash.ts → reemplaza el bash integrado
export default tool({
  description: "Restricted bash wrapper",
  args: { command: tool.schema.string() },
  async execute(a) { return `blocked: ${a.command}` },
})
```

! Mismo nombre = sobrescribe. Para solo **deshabilitar** una built-in, usa `permission`, no un tool.

● ARGUMENTOS · ZOD

```
# tool.schema (usa Zod por dentro)
query: tool.schema.string().describe("SQL")
# o Zod directo + objeto plano
import { z } from "zod"
export default {
  args: { param: z.string().describe("...") },
  async execute(args, context) { return "result" },
}
```

● CONTEXTO DE SESIÓN

```
async execute(args, context) {
  const { agent, sessionID, messageID,
    directory, worktree } = context
}
```

<code>context.directory</code>	Dir de trabajo de la sesión
<code>context.worktree</code>	Raíz del worktree de Git

● EJEMPLO · LÓGICA EN PYTHON DESDE UN TOOL TS

PYTHON

.OPENCODE/TOOLS/ADD.PY

```
import sys
a = int(sys.argv[1])
b = int(sys.argv[2])
print(a + b)
```

.OPENCODE/TOOLS/PYTHON-ADD.TS

```
import { tool } from "@opencode-ai/plugin"
import path from "path"
export default tool({
  description: "Add via Python",
  args: { a: tool.schema.number(), b:
tool.schema.number() },
  async execute(args, ctx) {
    const s = path.join(ctx.worktree,
".opencode/tools/add.py")
    return (await Bun.$`python3 ${s} ${args.a}
${args.b}`.text()).trim()
  },
})
```